Human Verification: `Containers.Tracking`

Compositional Tracking on Shipping Networks

Machine-generated from `lean/Tracking.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.Tracking` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

The file verifies Construction `con:tracking` (§4.3 of the paper): for any monoid M, a **compositional tracking** of M-data on a shipping network is in bijection with edge-labellings $G.E \to M$. The file also provides three concrete monoid instances and a cold-chain worked example.

Contents

1 Overview **2**

2 Bundled monoid **2**
2.1 Structure: `Monoid'` . . . 2

3 The Tracking structure **2**
3.1 Structure: `Tracking` . . . 2

4 Fold along a path **3**
4.1 Definition: `foldPath` . . . 3
4.2 Theorem: `foldPath_append` . . . 3

5 The bijection **3**
5.1 Definition: `Tracking.ofEdge` . . . 3
5.2 Definitions: `singleEdge` and `Tracking.toEdge` . . . 4
5.3 Theorems: `Tracking.ofEdge_toEdge` and `Tracking.toEdge_ofEdge` . . . 4
5.4 Definition: `trackingBijection` . . . 5

6 Monoid instances **5**
6.1 Definition: `maxTempMonoid` . . . 5
6.2 Definition: `coldChainMonoid` . . . 5
6.3 Definition: `licListMonoid` . . . 6

7 Cold-chain worked example **6**
7.1 Example: warm last-mile leg breaks the cold chain . . . 6

8 Translation conventions **7**

1 Overview

A tracking assigns a monoid value $d(v, \rho)$ to every route ρ out of node v, compatibly with the directed-container structure:

- **T-root**: $d(v, \mathrm{root}(v)) = 1$ (empty route gets unit);
- **T-shift**: $d(v, \mathrm{shift}(v, \rho, \sigma)) = d(v, \rho) \cdot d(\mathrm{sub}(v, \rho), \sigma)$ (concatenation becomes product).

The main theorem says this is exactly an edge-labelling: fold the label along the path.

2 Bundled monoid

2.1 Structure: `Monoid'`

Definition 2.1 (Bundled monoid)**.** A **bundled monoid** M consists of a carrier type M.M, a binary operation M.mul, a unit M.one, and proofs of associativity, left-unit, and right-unit.

Lean.

```
structure Monoid' where
  M         : Type
  mul       : M → M → M
  one       : M
  mul_assoc : ∀ a b c, mul (mul a b) c = mul a (mul b c)
  one_mul   : ∀ a, mul one a = a
  mul_one   : ∀ a, mul a one = a
```

Haskell.

```
-- Standard Haskell typeclass:
class Monoid m where
  mempty  :: m
  mappend :: m → m → m
  -- laws: mappend mempty x = x
  --       mappend x mempty = x
  --       mappend (mappend x y) z = mappend x (mappend y z)
```

3 The Tracking structure

3.1 Structure: `Tracking`

Definition 3.1 (Compositional tracking)**.** A **compositional tracking** of `Mon`-data on the network $\mathcal{N}(G)$ is a structure consisting of:

- a function $d : (v : G.V) \to \mathcal{N}(G).\mathrm{Pos}(v) \to M$,
- proof of **T-root**: $d(v, \mathcal{N}(G).\mathrm{root}(v)) = 1$ for all v, and
- proof of **T-shift**: for all v, ρ, σ, $d(v, \mathcal{N}(G).\mathrm{shift}(v, \rho, \sigma)) = d(v, \rho) \cdot d(\mathcal{N}(G).\mathrm{sub}(v, \rho), \sigma)$.

Lean.

```
structure Tracking (G : Digraph) (Mon : Monoid') where
  d       : (v : G.V) → (network G).Pos v → Mon.M
  d_root  : ∀ v, d v ((network G).root v) = Mon.one
  d_shift : ∀ (v : G.V) (rho : (network G).Pos v)
              (sigma : (network G).Pos ((network G).sub v rho)),
      d v ((network G).shift v rho sigma) =
        Mon.mul (d v rho) (d ((network G).sub v rho) sigma)
```

Haskell.
```haskell
data Tracking g m = Tracking
  { trackVal   :: Node g → Route g → m
  -- laws (not enforced by the type):
  -- trackVal v (root v) = mempty
  -- trackVal v (shift v rho sigma) =
  --   mappend (trackVal v rho) (trackVal (sub v rho) sigma)
  }
```

4 Fold along a path

4.1 Definition: `foldPath`

Definition 4.1 (Path fold)**.** Given an edge-labelling $f : G.E \to M$, the **fold** of f along a path $\rho : \mathrm{Path}_G(v, w)$ is:

$$\mathrm{foldPath}(M, f, \mathrm{nil}(v)) = 1, \qquad \mathrm{foldPath}(M, f, \mathrm{cons}(e, \rho)) = f(e) \cdot \mathrm{foldPath}(M, f, \rho).$$

Lean.
```lean
def foldPath {G : Digraph} (Mon : Monoid') (f : G.E → Mon.M) :
    {v w : G.V} → Path G v w → Mon.M
  | _, _, .nil _     ⇒ Mon.one
  | _, _, .cons e p ⇒ Mon.mul (f e) (foldPath Mon f p)
```

Haskell.
```haskell
foldPath :: Monoid m ⇒ (e → m) → [e] → m
foldPath _ []     = mempty
foldPath f (e:es) = mappend (f e) (foldPath f es)
-- equivalently: foldMap f
```

4.2 Theorem: `foldPath_append`

Theorem 4.2 (Fold distributes over concatenation)**.** For all edge-labellings f and paths ρ, σ:

$$\mathrm{foldPath}(M, f, \rho +\!\!+ \sigma) = \mathrm{foldPath}(M, f, \rho) \cdot \mathrm{foldPath}(M, f, \sigma).$$

Lean.
```lean
theorem foldPath_append {G : Digraph} (Mon : Monoid') (f : G.E → Mon.
    M) :
    ∀ {v w u : G.V} (p : Path G v w) (q : Path G w u),
    foldPath Mon f (p.append q) =
      Mon.mul (foldPath Mon f p) (foldPath Mon f q)
```

Haskell.
```haskell
-- foldPath f (p ++ q) = mappend (foldPath f p) (foldPath f q)
-- This is the standard homomorphism property of foldMap.
```

5 The bijection

5.1 Definition: `Tracking.ofEdge`

Definition 5.1 (Edge-labelling → tracking)**.** Given an edge-labelling $f : G.E \to M$, the induced tracking has $d(v, (w, \rho)) := \mathrm{foldPath}(M, f, \rho)$. T-root holds because foldPath of the empty path is 1; T-shift holds by `foldPath_append`.

Lean.

```
def Tracking.ofEdge {G : Digraph} {Mon : Monoid'} (f : G.E → Mon.M) :
    Tracking G Mon where
  d _ rho    := foldPath Mon f rh ∘ 2
  d_root _ := rfl
  d_shift _ rho sigma := foldPath_append Mon f rh ∘ 2 sigma.2
```

Haskell.

```
ofEdge :: Monoid m ⇒ (e → m) → Tracking g m
ofEdge f = Tracking { trackVal = \_ route → foldPath f (path route) }
```

5.2 Definitions: `singleEdge` and `Tracking.toEdge`

Definition 5.2 (Single-edge position and restriction)**.** The **single-edge position** at the source of e is the route $(G.\mathrm{tgt}(e), \mathrm{cons}(e, \mathrm{nil})) \in \mathcal{N}(G).\mathrm{Pos}(G.\mathrm{src}(e))$.

Given a tracking t, the **restriction to edges** is $\mathrm{toEdge}(t)(e) := t.d(G.\mathrm{src}(e), \mathrm{singleEdge}(G, e))$.

Lean.

```
def singleEdge (G : Digraph) (e : G.E) : (network G).Pos (G.src e) :=
  ⟨G.tgt e, Path.cons e (.nil (G.tgt e))⟩

def Tracking.toEdge {G : Digraph} {Mon : Monoid'} (t : Tracking G Mon)
    :
    G.E → Mon.M :=
  fun e ⇒ t.d (G.src e) (singleEdge G e)
```

Haskell.

```
singleEdge :: Digraph v e → e → Route v e
singleEdge g e = (tgt g e, [e])

toEdge :: Tracking g m → e → m
toEdge t e = trackVal t (src g e) (singleEdge g e)
```

5.3 Theorems: `Tracking.ofEdge_toEdge` and `Tracking.toEdge_ofEdge`

Theorem 5.3 (Bijection round trips)**.** The maps `toEdge` and `ofEdge` are mutually inverse:

1. **Left inverse**: For every tracking t, $\mathrm{ofEdge}(\mathrm{toEdge}(t)) = t$. (Uses T-root and T-shift induction along the path: `foldPath_eq_tracking`.)
2. **Right inverse**: For every edge-labelling f, $\mathrm{toEdge}(\mathrm{ofEdge}(f)) = f$. (Immediate: fold of the single-edge path is $f(e) \cdot 1 = f(e)$.)

Lean.

```
theorem Tracking.ofEdge_toEdge {G : Digraph} {Mon : Monoid'}
    (t : Tracking G Mon) : Tracking.ofEdge (t.toEdge) = t

theorem Tracking.toEdge_ofEdge {G : Digraph} {Mon : Monoid'}
    (f : G.E → Mon.M) : (Tracking.ofEdge f).toEdge = f
```

Haskell.

```
-- ofEdge . toEdge = id    (for trackings)
-- toEdge . ofEdge = id    (for edge-labellings)
```

5.4 Definition: `trackingBijection`

Definition 5.4 (Tracking bijection (Main Theorem))**.** The pair (toEdge, ofEdge) constitutes a bijection

$$\text{Tracking}(G, M) \;\cong\; (G.E \to M).$$

Bundled as a `Bijection` record with the two round-trip proofs.

Lean.

```
def trackingBijection (G : Digraph) (Mon : Monoid') :
    Bijection (Tracking G Mon) (G.E → Mon.M) where
  toFun     := Tracking.toEdge
  invFun    := Tracking.ofEdge
  left_inv  := Tracking.ofEdge_toEdge
  right_inv := Tracking.toEdge_ofEdge
```

Haskell.

```
-- The bijection (toEdge, ofEdge) establishes:
-- Tracking G M  ~=  (Edge G → M)
trackingBijection :: Iso (Tracking g m) (e → m)
trackingBijection = Iso toEdge ofEdge
```

6 Monoid instances

6.1 Definition: `maxTempMonoid`

Definition 6.1 (Max-temperature monoid)**.** The **max-temperature monoid** ({Cold, Warm}, max, Cold) is the idempotent monoid for peak-temperature tracking: `Warm` dominates `Cold` (absorbs it), and `Cold` is the unit. A single warm leg along any route dominates all cold legs; no subsequent refrigeration can undo it.

Lean.

```
inductive MaxTemp | Cold | Warm
  deriving DecidableEq, Repr

def maxTempMonoid : Monoid' where
  M         := MaxTemp
  mul       := maxTempMul
  one       := .Cold
  one_mul   := fun x ⇒ by cases x <;> rfl
  mul_one   := fun x ⇒ by cases x <;> rfl
  mul_assoc := fun x y z ⇒ by cases x <;> cases y <;> cases z <;> rfl
```

Haskell.

```
data MaxTemp = Cold | Warm deriving (Eq, Show)

instance Semigroup MaxTemp where
  Cold <> x = x
  Warm <> _ = Warm

instance Monoid MaxTemp where
  mempty = Cold
```

6.2 Definition: `coldChainMonoid`

Definition 6.2 (Cold-chain monoid)**.** The **cold-chain monoid** (Bool, &&, true): true means the chain is intact; false means it is broken. Once one leg breaks the chain, no subsequent leg can repair it.

Lean.

```
def coldChainMonoid : Monoid' where
  M         := Bool
  mul       := (ů && ů)
  one       := true
  one_mul   := Bool.true_and
  mul_one   := Bool.and_true
  mul_assoc := Bool.and_assoc
```

Haskell.

```
-- (All, getAll) from Data.Monoid: mempty = True, mappend = (&&)
newtype ColdChain = ColdChain Bool

instance Monoid ColdChain where
  mempty = ColdChain True
  mappend (ColdChain a) (ColdChain b) = ColdChain (a && b)
```

6.3 Definition: `licListMonoid`

Definition 6.3 (Licence-accumulation monoid)**.** For any licence type L, the **licence-list monoid** $(\mathrm{List}(L), ++, [])$ accumulates the union (as a list) of licences required by each leg along a route.

Lean.

```
def licListMonoid (L : Type) : Monoid' where
  M         := List L
  mul       := (ů ++ ů)
  one       := []
  one_mul   := List.nil_append
  mul_one   := List.append_nil
  mul_assoc := List.append_assoc
```

Haskell.

```
-- Standard list monoid: mempty = [], mappend = (++)
type LicList l = [l]
```

7 Cold-chain worked example

7.1 Example: warm last-mile leg breaks the cold chain

Proposition 7.1 (Warm leg breaks the chain)**.** In a 2-node graph (`Hub` → `Shop`) with a single leg `lastMile` labelled false (warm), the tracking value of the route Hub → Shop is false, and the empty route is true.

Lean.

```
-- The cold-chain tracking induced by warmLabel (lastMile ⇒ false):
example : coldTracking.d Hub
    ⟨Shop, Path.cons (G := coldGraph) .lastMile
                     (.nil (G := coldGraph) Shop)⟩ = false

-- The empty route is intact:
example : coldTracking.d Hub ⟨Hub, .nil (G := coldGraph) Hub⟩ = true
```

Haskell.

```
warmLabel :: Arc → Bool
warmLabel LastMile = False

coldTracking :: Tracking ColdGraph Bool
coldTracking = ofEdge warmLabel
```

```
-- trackVal coldTracking Hub [LastMile] = False
-- trackVal coldTracking Hub []         = True
```

8 Translation conventions

1. **Bundled monoid.** Lean uses a record `Monoid'` instead of a typeclass, matching the bundled style of the rest of the formalisation. In Haskell, the standard `Monoid` typeclass plays the same role.
2. **T-root and T-shift as data.** The tracking laws are bundled into the `Tracking` structure as proof fields, not stated as separate theorems. In Haskell they are law comments; QuickCheck could test them.
3. **Proof bodies omitted.** Only definitions and theorem *statements* are translated. Lean checks the proofs; the human verifier checks that the *statements* match the intended mathematics.